§2 Formal definition of an ACS — part 1 of 5

Reference: sec:acs_def

Source: paper/section2_definition.tex, lines 1–720 (§2 opening to §2.4), at commit e6e12cd. The Japanese manuscript is authoritative. §2 is summarized in five parts: part 1 (this file): scope and conventions, external background and type separation, primitive and base functions, the 7-tuple; part 2: structural self, temporal self-identity, functional termination and collapse (§2.5); part 3: the required environment space and CR-1 (§2.6); part 4: state transition, the persistence evaluation function, feasibility and blockage (§2.7–§2.8); part 5: CR-2 and standard maps, the internal model and its update, cognitive processing capacity, and the list of conventions, requirements and assumptions (§2.9–§2.13).

---

§2 defines the ACS previewed in §1 as the formal basis of a single ACS to which later sections refer. It does not restate the positioning, overall scope, general rules of the three classes or motivation of §1; it confirms only what is needed to define the basic types of an ACS, its constitutive requirements, well-formed realization, feasibility, primitive functions, internal model and update structure. It does not treat the origin, implementation history or evolution of an ACS; it gives the abstract structure under which an object is described as an ACS in the paper's sense. The policy "claim strength ≤ proof strength" is adopted throughout. §2 defines blockage, functional termination and collapse (def:blockage_sec2, def:functional_termination, def:collapse), and lists at its end the matters it does not claim on its own.

2.1 Scope and common conventions

Reference: subsec:s2_scope

Separation from §1 (subsubsec:s2_s1_boundary). "Definition" here means fixing the types and conditions used to describe the class of objects. General ethical evaluation, empirical identification of implementations, and claims of superiority over existing theories are not among the conclusions of §2. Classifications A, B and C of §1 are not rules that change the number of components of the 7-tuple (rem:abc_classification_rules).

Three statuses (subsubsec:s2_requirement_status). Conventions, constitutive requirements and assumptions are treated as having different statuses.

- **Convention.** An agreement specifying how the paper writes formal objects: the Type Separation Convention TSC (§2.2) and the convention on levels of description (§2.4). A convention is not a functional capacity of the ACS itself.
- **Constitutive requirement (CR).** A condition specifying what the paper calls an ACS: required-environment consistency CR-1 (§2.6) and possession of primitive and base functions CR-2 (§2.9). CR-1 and CR-2 cannot be checked mechanically from the 7-tuple; they are semantic conditions on its realization, and they are not counted as ordinary mathematical assumptions.
- **Assumption.** A condition used to discuss further results about defined objects. Each assumption carries an introduction status (A, S, O) and a strength (weak, medium, strong). The assumptions of §2 are listed in §2.13.

TSC is therefore not read as a functional capacity of an ACS, and CR-1 and CR-2 are not counted as added mathematical assumptions. def:acs treats as an ACS a structure that has a realization satisfying CR-1 and CR-2 under the convention TSC.

Phase 1 boundary (subsubsec:s2_phase1_boundary). §2 deals with the structure of a single ACS that is deterministic, in discrete time, with a compact internal state space. Time is $t \in \mathbb{N}_0$ and the state is updated by $t \mapsto t + 1$. The external environment state is split into an exogenous background component and an action-response component. The sequence of exogenous background components is treated as a realized sequence given in deterministic analysis; its generating law or frequency is not specified. The action-response component can respond to the ACS's actions according to an environment-side transition map (def:env_realization). Stochastic extension, continuous time, generalization to non-compact spaces, empirical procedures for identifying required environmental factors, dynamic reorganization of the self–

environment boundary, and structural application to social organizations are separate from the formal definitions of §2 and are left to later sections or Phases.

2.2 External background and self–environment type separation

Reference: subsec:wellformedness

"External" is not a universal classification of the world; it is relative to the boundary of the ACS individual considered. Where several individuals must be distinguished, the external environment state space relative to Σ_i is written $E_{\mathrm{ext},i}$; in §2 the index is omitted.

E_{ext} is the type of candidate external environment states that can be used to describe the ACS's state transition, sensing and other external inputs. Its elements are $e \in E_{\mathrm{ext}}$, and the realized external environment state at time t is $e_t \in E_{\mathrm{ext}}$. E_{ext} is not an independent component of the 7-tuple but the external background type relative to which the 7-tuple is defined. In Phase 1, E_{ext} is treated as a topological space, as the basis for stating the continuity of f. Its compactness is not assumed in general; propositions that need it state it as an added assumption where used.

The basic conditions supporting the definition of an ACS are of two kinds. A convention is an agreement the paper adopts for description, not a condition the ACS must meet. A constitutive requirement (CR) is a condition the 7-tuple structure must meet to be an ACS in the paper's sense. Both are distinguished from ordinary assumptions and are not in the list of assumptions.

Type Separation Convention TSC (`conv:TSC`). The internal state space X of Σ_i and the external environment state space E_{ext} relative to it are treated as different types in the paper's set representation. In Phase 1 a tagged disjoint representation is adopted and one writes $X \cap E_{\mathrm{ext}} = \emptyset$. This is a constructive convention so as not to confuse, in an untagged set representation, mathematical objects representing internal states with those representing external environment states. It does not claim that the matter making up the inside of an ACS and the matter making up its environment are ontologically of different kinds. Even if the same representation (for example the same sequence of numbers) can appear both as an internal state and as an external environment state, the tagged objects are distinct.

- **Character of TSC (`rem:TSC_character`).** Even with an untagged representation where $X \cap E_{\mathrm{ext}} \neq \emptyset$, the type of the state transition map can be defined by specifying internal-state and external-state arguments as different sorts. TSC is therefore not indispensable to the mathematical well-formedness of the map; it is a convention the paper adopts to separate the types of self and environment explicitly. The tagged disjoint representation is a means of implementing it; what TSC requires is the type separation itself. If Phase 2 treats a fluid or reorganized self–environment boundary, the handling of TSC will be stated anew.
- **Individual relativity (`rem:Eext_relative`).** Because E_{ext} is relative to the boundary of the ACS, different individuals Σ_i, Σ_j are not required to share it. For Σ_i, another individual Σ_j can appear as part of Σ_i's environment, while Σ_j's own internal state belongs to Σ_j's state space. Phase 1 introduces no transformation maps or common ambient space between the E_{ext} of different individuals; formulations that need such correspondences are left to Phase 2.

Definition (`def:Eext_decomposition`): decomposition of the external environment state space. E_{ext} is written as the product of two topological spaces, with the product topology:

$$E_{\mathrm{ext}} = E^{\mathrm{bg}} \times O, \qquad O := \prod_{a \in \mathcal{A}_O} O_a.$$

Elements are $e = (b, o)$, and the realized state at t is $e_t = (b_t, o_t)$.

1. E^{bg} is the space of **exogenous background components**. Its element b represents external conditions that do not respond to the ACS's actions.
2. $\mathcal{A}_O$ is a finite set, fixed over time; its elements a are called **slots**.
3. Each O_a is a topological space containing a distinguished element $\perp_a$, the **absent state** of slot a. O is the space of **action-response components**, and the a-th coordinate of $o \in O$ is $o_a \in O_a$.

If $\mathcal{A}_O = \emptyset$, O is a one-point set; E^{bg} being a one-point set is not excluded either.

Any external environment state space can be put in this form by taking $\mathcal{A}_O = \emptyset$ and $E^{\mathrm{bg}} = E_{\mathrm{ext}}$, so the decomposition does not narrow the range of ACSs considered. Whether to have action-response components, and which components to assign to slots, is a choice in describing the system. TSC is kept after the decomposition: $X \cap (E^{\mathrm{bg}} \times O) = \emptyset$.

Reading of slots and action-response components (`rem:Eext_decomposition_reading`).

- (a) A slot is not an individual thing but a position for describing a component of the environment that can respond to the ACS's actions. A thing appearing or disappearing is represented by o_a moving between $\perp_a$ and other values. The same slot may be reused at different times for different things; the paper does not track the identity of things from the sequence of slot states.
- (b) Action-response components are not limited to things. Any external quantity that can respond to the ACS's actions may be assigned to a slot, for example an accumulated quantity (such as the ACS's emissions accumulated nearby) or a local field (such as the surrounding temperature changed by the ACS's heat).
- (c) The distinction between E^{bg} and O is a descriptive distinction by whether a component can respond to the ACS's actions, not an ontological division of the external world.
- (d) "Object" and "target object" are used in §4 as words concerning discrimination internal to the ACS. The slot index set $\mathcal{A}_O$ is a theory-level index and is identified neither with the ACS's internal object representations nor with the set of external objects $\mathcal{Y}_{-i}$ of §6.

Notation: E^{bg} is unrelated to the set of required environmental factors E (§2.6). O is unrelated to the symbol $\mathcal{O}$ for organizations in §8. The absent state $\perp_a$ is a different symbol from $\perp$ used in §4 for the absence of a local protective response.

2.3 Primitive functions and base functions

`Reference: subsec:primitive_functions`

Before the 7-tuple, §2.3 fixes the abstract functional divisions the paper uses to describe the functions of an ACS, and the base functions that support them. Only their meaning is given here; basic functions, CR-2 and the typed standard maps are given in §2.9.

Definition (`def:primitive_functions`): primitive functions (classification A). The functional responsibilities of an ACS from obtaining information to producing a response are described in four abstract divisions, each called a **primitive function**:

- **Sensing:** obtaining information from one's own internal state and from the external environment state.
- **Discrimination:** distinguishing the information obtained and forming internal representations usable by later evaluation and action.
- **Evaluation:** treating internal representations, one's current state, candidate responses or other objects of evaluation non-equivalently, in the light of internal criteria concerning one's own state. It is not required that all of these be objects of evaluation.
- **Action:** producing a response according to at least one of the available internal representations, evaluations or the current state. It is not required that the response depend on all of these.

"Primitive" means adopted, in the paper's functional description, as a unit that is not defined from other concepts and not divided further. It does not mean biological or evolutionary origin or age, or physical indivisibility.

Primitive functions are abstract divisions above the concrete functions an individual has (basic functions, `def:basic_functions`); they do not refer to particular physical parts, computational modules, maps or divisions of an implementation. Several functions may jointly carry the responsibility of one division, and one function may take part in the responsibilities of several

divisions. The order of the four divisions only shows a typical pattern of informational dependence; it does not mean sequential execution at each time or a fixed processing sequence.

"Self" in this definition is used informally for the system described as an ACS; its formal counterpart is fixed in §2.5. The internal criteria in evaluation are criteria of selection that the ACS itself carries; they are different from the persistence evaluation function Π of §2.8. The paper does not require the ACS to hold or compute Π internally (`subsec:description_levels_global`).

Definition (`def:base_functions`): base functions (classification A). Functions that support the activity of the functions carrying the responsibilities of the primitive divisions, and that keep their carriers in existence, are called **base functions**. They may include supply and transport of matter and energy, metabolism, repair, adjustment of the substrate state, and others. Base functions are not among the primitive divisions and are not treated as "units not divided further" in the same sense. They do not carry the responsibilities from obtaining information to producing a response; they maintain the conditions under which the functions carrying those responsibilities can operate. One function may take part in a primitive division's responsibility in one aspect and work as a base function in another.

Bodily regulation that senses an internal state, discriminates and evaluates its difference from a set criterion, and acts internally to adjust the state can be treated as the case where the four primitive responsibilities work on the internal state; no division is added for it. Work that keeps the carriers of functions in existence, such as transport of matter or metabolism, is treated as a base function. When one process takes part in both, the paper does not require assigning it to one of them; the aspect that carries the responsibility of control and the aspect that keeps carriers in existence may be described separately for the same process. Which aspect is described under which division or as a base function is a choice in describing the system; Phase 1 gives no general criterion.

Levels of description (`rem:primitive_vs_principles`). Primitive functions and base functions are concepts at the level of functions. The three principles of §3 (SM, EU, FR) are constraint axes on an existing ACS; the paper states no holding or failing of a principle itself, and what has a truth value is the time-indexed satisfaction predicate of each principle. No primitive division is mapped one-to-one to a particular principle, and no function dedicated to a principle is posited. The paper states neither of the following as a general consequence: that a time-indexed satisfaction predicate is true because a basic function or base function concerning a primitive division is operating; or that a particular basic function or base function has stopped because a time-indexed satisfaction predicate is false. The correspondence between primitive functions and the structural functions related to the three principles (`def:principle_related_functions`) is treated in §3.

2.4 The ACS 7-tuple and basic types

`Reference: subsec:acs_7tuple`

A 7-tuple is the ordered list of the seven basic components describing an ACS. §2.4 fixes the basic type of each component; the meaning of the structural self is given in §2.5, the semantics of required environments in §2.6, and the constitutive requirement on primitive functions in §2.9.

Definition (`def:acs`): autonomous cognitive system. Relative to an external environment state space E_{ext}, under the convention TSC, the following 7-tuple structure that has a realization satisfying CR-1 (required-environment consistency, §2.6) and CR-2 (possession of primitive and base functions, §2.9) is called an **autonomous cognitive system (ACS)**:

$$\Sigma = (X, U, E, f, \Pi, M, \text{Update}).$$

1. **State space X.** The space of internal states the ACS can take; the internal state at t is $x_t \in X$. In Phase 1, $X \subseteq \mathbb{R}^n$ is treated as compact (`ass:compact_spaces`). X is not the matter making up the ACS but a state space formally representing the internal states it can take while realized as the same individual.
2. **Action space U.** The space of effective actions the ACS can select; the effective action at t is $u_t \in U$. In Phase 1, $U \subseteq \mathbb{R}^m$ is treated as compact (`ass:compact_spaces`). A label structure may be introduced inside U to distinguish actions on the external environment, actions on internal cognitive states, and selective waiting (`rem:action_space_structure`).

Even if later mechanism descriptions distinguish several internal command sources or local responses, the u_t fed into f is the effective action after these are resolved by the rules of the implementation.

3. **Required environment space E.** The set representing the environmental factors indispensable for the ACS to persist as the same individual. Persistence here means not that the 7-tuple exists as a mathematical object but that the well-formed realization as the same individual continues (§2.5). In Phase 1 each required environmental factor is a binary satisfaction map on pairs of an internal state and an external environment state:

$$E \subseteq \{0,1\}^{X \times E_{\text{ext}}},$$
$$E \neq \emptyset,$$
$$\eta : X \times E_{\text{ext}} \to \{0,1\} \quad (\eta \in E).$$

   $\eta(x,e) = 1$ means the factor is satisfied for that pair, and $\eta(x,e) = 0$ that it is not. No continuity or other regularity is imposed on η in Phase 1; propositions needing it state an added assumption. Because elements of E are not external environment states, the realized external environment state is written $e_t \in E_{\text{ext}}$, not $e_t \in E$. The constitutive requirement CR-1, which constrains why each element of E is called "required", the external dependence and joint satisfaction of required factors, and the non-uniqueness of the factor decomposition are given in §2.6.

4. **State transition map f.** The map giving the next internal state from the internal state, the effective action and the realized external environment state:

$$f : X \times U \times E_{\text{ext}} \to X, \qquad x_{t+1} = f(x_t, u_t, e_t).$$

   In Phase 1, f is treated as continuous (`ass:f_continuity`). The component f is defined on the whole of E_{ext}, not as a restriction to the required environment space, so that external inputs that are not essential for persistence but affect the transition, neutral inputs and harmful inputs can be described under the same type. The decomposition of f into an autonomous functional process and a directed process is treated later (`ass:f_decomposition`).

5. **Persistence evaluation function Π.** A real-valued function $\Pi : X \to \mathbb{R}$ evaluating the feasibility of internal states. As the basic convention, a state x_t is interpreted as feasible when $\Pi(x_t) \geq 0$. The feasible set $K = \{x \in X \mid \Pi(x) \geq 0\}$ is not an independent component of the 7-tuple; it is derived from Π (`def:K`).

6. **Internal model structure M.** The internal model structure by which the ACS processes information about the external environment, its own state and its possibilities of action, and uses it to form responses. The paper takes $M = (M_1, M_2, M_3)$ and treats it as the Triadic Information-Processing Architecture (TIPA), with M_1 the information-integration mechanism, M_2 the adaptive-evaluation mechanism and M_3 the response-action mechanism. The space of internal-model states is $\mathcal{M}$, and the internal-model state at t is $M_t \in \mathcal{M}$. M (the mechanism structure), $\mathcal{M}$ (the space its states take) and M_t (the state at t) are distinguished. The minimal structure of the internal model is in §2.10 and the details of TIPA's standard functional form are in §4.

7. **Internal-model update operator** Update**.** The operator updating the internal-model state $M_t \in \mathcal{M}$ according to the realized internal state and realized external environment state. Its comprehensive type in the 7-tuple is

$$\text{Update} : \mathcal{M} \times X \times E_{\text{ext}} \to \mathcal{M}.$$

   The detailed component-wise update types are given in §2.11 (`def:update_operator`), and the relation between the abstract and detailed types is set out in `rem:update_type_two_layers`.

With these seven components, an ACS is described with the following kept distinct: possible internal states, possible actions, indispensable dependence for persistence on the external world, state transition, feasibility evaluation, internal model structure, and internal-model update. Where several individuals must be distinguished, one writes $\Sigma_i = (X_i, U_i, E_i, f_i, \Pi_i, M_i, \text{Update}_i)$ with external environment state space $E_{\text{ext},i}$; in §2 these indices are omitted except where needed.

Convention on levels of description (`conv:structure_levels`). Symbols concerning an ACS are written at three levels. This convention is a descriptive agreement, neither a functional capacity of the ACS nor an assumption restricting the range of later propositions.

- **Abstract structure type.** The type of 7-tuple structure fixed by `def:acs`. It is not required to treat the abstract structure type as a set; so an individual Σ_i is not written as an element of it, but as "a realization of the abstract structure type".
- **Individual Σ_i.** The 7-tuple of one ACS individual identified as a realization of the abstract structure type. In Phase 1 the individual's 7-tuple is treated as a formal structure independent of time (`def:s_struct`). What can change with time are time-indexed values such as the state $x_{i,t} \in X_i$ and the internal-model state. In particular, the mechanism configuration of M_i is fixed; changes in the state $M_{c,t}$ of each mechanism, in functional memory contents and in other time-indexed values are described as internal-model update and as the change between time steps of §5. Change over time of the mechanism configuration itself is not treated.
- **Time-indexed values.** Quantities depending on the realization of an individual (state and environment sequences) are not written as functions taking the structure as input; the individual index and the time index are put in subscripts. A time-independent map or correspondence and its value at a time are written in two stages. For example, the correspondence $A_i(x, e)$ and the time value $A_{i,t} := A_i(x_{i,t}, e_{i,t})$ are distinguished (`def:at`). Symbols for time values are used only where defined in the text.

For the internal model, the mechanism structure M (a component of the 7-tuple), its state space $\mathcal{M}$ and the internal-model state $M_t \in \mathcal{M}$ at time t are distinguished (§2.10). The c-th mechanism M_c of TIPA and its state $M_{c,t} \in \mathcal{M}_c$ at time t also differ in type according to whether the time index is present. The description of a case where an individual's 7-tuple itself changes over time is a task for Phase 2 (L16 in `subsec:limitations_list`).

Supplementary remarks on the basic types

- **Position of K (`rem:K_position`).** The feasible set K is not held as an independent component of the 7-tuple. It is a derived object, obtained from Π and the paper's convention for judging feasibility as $K = \{x \in X \mid \Pi(x) \geq 0\}$. The construction of Π, the formal definition of K and their relation are given in §2.8.
- **Structure of U (`rem:action_space_structure`).** Inside the compact set U a label structure is identified: $U_{\text{ext}} \subseteq U$ (actions acting directly on the external environment), $U_{\text{int}} \subseteq U$ (actions intentionally changing internal cognitive states), and $u_\emptyset \in U$ (selective waiting). These are subsets and an element of U identified as labels and do not change the compactness of U. $u_\emptyset$ does not denote the absence of an action; it is a regular element of U and, like other elements, is an action argument of f. Effects on the topological structure are a task for Phase 2. The relation of $u_\emptyset$ to the autonomous functional process when it is selected is fixed by the later decomposition of the transition (`ass:f_decomposition`). The effect of an action on action-response components is described not by the label U_{ext} but by the action difference (`def:action_difference`). External forcing is not an action of the ACS; it is represented as a change of state entering f through the external environment state e_t. An action u_t of the ACS and external forcing can coexist at the same time. Classes of actions are treated in `def:action_classes` of §4, and the fact that the criterion of a purpose is read out from carriers internal to the ACS in `prop:purpose_self_attribution` of §4.
- **Two layers of the type of** Update **(`rem:update_type_two_layers`).** The type $\mathcal{M} \times X \times E_{\text{ext}} \to \mathcal{M}$ in the 7-tuple is a comprehensive abstract type. The detailed component-wise types (`def:update_operator`, §2.11) may be defined as a family of maps including additional arguments available in an implementation. The abstract and detailed types therefore need not be the same type directly; they are treated consistently as a difference in level of abstraction (under stated assumptions). Additional arguments in the detailed types are intermediate values or auxiliary information obtained in the same step or by preceding processing, and are not by themselves a reason to add an independent component to the 7-tuple.

Non-redundancy of the 7-tuple

Reference: rem:tuple_nonredundancy

The paper adopts the 7-tuple as the basic components of the Phase 1 definition of an ACS. Under the paper's representational system, they carry different structural roles: possible internal states, possible actions, necessity for persistence with respect to the external world, state transition, feasibility evaluation, internal model structure, and internal-model update.

In particular, the required environment space E is neither the type E_{ext} of realized external environment states nor a subset of it. It is primitive constitutive information representing, as a set of required environmental factors, the indispensable dependence for persistence that the ACS has on the external world. In Phase 1, E is not defined as a quantity derived uniquely from X, f, Π or other components. The paper gives no rule fixing uniquely from f which of the environmental differences described by f are constitutively indispensable for the persistence of the same ACS, nor any rule fixing uniquely from Π the granularity of decomposition into required factors. The semantic constraints on being required and the non-uniqueness of decomposition are treated in §2.6.

Accordingly, under the paper's representational requirement of keeping distinct internal state, action, environment required for persistence, state transition, feasibility evaluation, internal model and internal-model update, the seven components are regarded as non-redundant in the sense that deleting any one of them leaves at least one structural role no longer held explicitly.

The paper does not, however, claim absolute minimality against arbitrary mathematical reencodings. For example, using an extended state space $Y := X \times \mathcal{M}$ and an extended transition map, it is formally possible to absorb the internal-model state and the update operator into the state transition. In that case, the distinctions the paper keeps explicit — between internal state and internal-model state, and between state transition and internal-model update — are lost. What this remark states is non-redundancy within the present framework, which preserves these separations of role.